考试中心填写：

___年 ___月___日
考试用

湖南大学课程考试试卷

课程名称：计算机系统　　；试卷编号：A；考试时间：120 分钟

姓名：　学号：　专业班级：

题号	一	二	三	四	五	六	七	八	九	十	总分
应得分	10	15	15	10	25	25					100
实得分											
评卷人											评分：

（如无特别说明，所示汇编代码均在 32 位机小端环境下编译得到）

1.简答题（10 分）

请用 32 位补码整数和 IEEE 754 单精度浮点数格式分别表示 2049 这个值（结果请用 16 进制表示）。（7 分）

请问两种表示方法二进制序列有相同的部分吗？为什么？（3 分）

2.程序填空题（15 分，每空 3 分）

如下是一个 c 语言程序及其对应的汇编代码，请参照汇编代码，完成 c 程序的空缺部分。

c 语言程序：

```
int main()
{
    int i=0,j=1,k=2,loop=3;
    int* p=&i;
    int arr[3]={9,   (1)   ,31};
    scanf("%d,%d",&i,&j);
    p=   (2)   ;
    k=   (3)   ;
    for(loop=0;loop<2;   (4)   )
    {
        (5)   ;
    }
    return 0;
}
```

汇编代码如下：

```
main:
pushl  %ebp
movl   %esp, %ebp
```

```
    andl   $-16, %esp
    subl   $48, %esp
    movl   $0, 28(%esp)
    movl   $1, 32(%esp)
    movl   $2, 36(%esp)
    movl   $3, 40(%esp)
    leal   28(%esp), %eax
    movl   %eax, 44(%esp)
    movl   $9, 16(%esp)
    movl   $5, 20(%esp)
    movl   $31, 24(%esp)
    movl   $.LC0, %eax  //scanf 调用的字符串
    leal   32(%esp), %edx
    movl   %edx, 8(%esp)
    leal   28(%esp), %edx
    movl   %edx, 4(%esp)
    movl   %eax, (%esp)
    call   __isoc99_scanf
    leal   16(%esp), %eax
    addl   $4, %eax
    movl   %eax, 44(%esp)
    movl   44(%esp), %eax
    movl   (%eax), %eax
    movl   32(%esp), %edx
    addl   $1, %edx
    cmpl   %edx, %eax
    jle .L2
    movl   28(%esp), %eax
    addl   $3, %eax
    jmp .L3
.L2:
    movl   28(%esp), %eax
    addl   %eax, %eax
.L3:
    movl   %eax, 36(%esp)
    movl   $0, 40(%esp)
    jmp .L4
.L5:
    movl   36(%esp), %eax
    leal   (%eax,%eax), %edx
    movl   44(%esp), %eax
    movl   (%eax), %eax
    addl   %edx, %eax
    movl   %eax, 36(%esp)
```

```
    addl   $1, 40(%esp)
    addl   $4, 44(%esp)
.L4:
    cmpl   $1, 40(%esp)
    jle .L5
    movl   $0, %eax
    leave
    ret
```

3.程序填空题（第三空 3 分，其余三空每空 4 分，共 15 分）

如下是一个 c 语言程序及其对应的汇编代码，请参照汇编代码，完成 c 程序的空缺部分。

c 语言程序：

```
struct s
{
    int i;
    int j;
    struct s *pnext;
}*ps1,*ps2;
int f(int k, struct s * ps)
{
    if(   (1)   )
        return   (2)   ;
    else
        return k+ps->i+ps->j;
}
int main()
{
    ps1=(struct s*)malloc(sizeof(struct s));
    ps2=(struct s*)malloc(sizeof(struct s));
    ps1->i=1;
    ps1->j=2;
    ps2->i=3;
    ps2->j=4;
    ps1->pnext=NULL;
    ps2->pnext=NULL;
    __(3)__;
    ps1->j=__(4)__;
    printf("%d\n",ps1->j);
    return 0;
}
```

汇编代码如下：

```
f:
    pushl  %ebp
```

```
    movl   %esp, %ebp
    movl   12(%ebp), %eax
    movl   8(%eax), %eax
    testl  %eax, %eax
    je .L2
    movl   12(%ebp), %eax
    movl   (%eax), %eax
    movl   %eax, %edx
    imull  8(%ebp), %edx
    movl   12(%ebp), %eax
    movl   4(%eax), %eax
    imull  %eax, %edx
    movl   12(%ebp), %eax
    movl   8(%eax), %eax
    movl   (%eax), %ecx
    movl   12(%ebp), %eax
    movl   8(%eax), %eax
    movl   4(%eax), %eax
    imull  %ecx, %eax
    movl   %edx, %ecx
    subl   %eax, %ecx
    movl   %ecx, %eax
    jmp .L3
.L2:
    movl   12(%ebp), %eax
    movl   (%eax), %eax
    movl   %eax, %edx
    addl   8(%ebp), %edx
    movl   12(%ebp), %eax
    movl   4(%eax), %eax
    addl   %edx, %eax
.L3:
    popl   %ebp
    ret
main:
    pushl  %ebp
    movl   %esp, %ebp
    pushl  %esi
    pushl  %ebx
    andl   $-16, %esp
    subl   $16, %esp
    movl   $12, (%esp)
    call   malloc
    movl   %eax, ps1
```

```
movl   $12, (%esp)
call   malloc
movl   %eax, ps2
movl   ps1, %eax
movl   $1, (%eax)
movl   ps1, %eax
movl   $2, 4(%eax)
movl   ps2, %eax
movl   $3, (%eax)
movl   ps2, %eax
movl   $4, 4(%eax)
movl   ps1, %eax
movl   $0, 8(%eax)
movl   ps2, %eax
movl   $0, 8(%eax)
movl   ps2, %eax
movl   ps1, %edx
movl   %edx, 8(%eax)
movl   ps1, %ebx
movl   ps1, %esi
movl   ps2, %edx
movl   ps2, %eax
movl   (%eax), %eax
movl   %edx, 4(%esp)
movl   %eax, (%esp)
call   f
movl   %esi, 4(%esp)
movl   %eax, (%esp)
call   f
movl   %eax, 4(%ebx)
movl   ps1, %eax
movl   4(%eax), %edx
movl   $.LC0, %eax   //printf 调用的字符串
movl   %edx, 4(%esp)
movl   %eax, (%esp)
call   printf
movl   $0, %eax
leal   -8(%ebp), %esp
popl   %ebx
popl   %esi
popl   %ebp
ret
```

4.综合分析题（10 分）
一个简单的函数调用 C 代码经编译后得到的核心汇编代码如下：

```
0804841d <main>:
 804841d: 55                    push   %ebp
 804841e: 89 e5                 mov    %esp,%ebp
 8048420: 83 e4 f0              and    $0xfffffff0,%esp
 8048423: 83 ec 20              sub    $0x20,%esp
 8048426: c7 44 24 14 03 00 00
 804842d: 00                    movl   $0x3,0x14(%esp)
 804842e: c7 44 24 18 04 00 00
 8048435: 00                    movl   $0x4,0x18(%esp)
 8048436: 8b 44 24 18           mov    0x18(%esp),%eax
 804843a: 89 44 24 04           mov    %eax,0x4(%esp)
 804843e: 8b 44 24 14           mov    0x14(%esp),%eax
 8048442: 89 04 24              mov    %eax,(%esp)
 8048445: e8 1a 00 00 00        call   8048464 <f1>
 804844a: 89 44 24 1c           mov    %eax,0x1c(%esp)
 804844e: 8b 44 24 1c           mov    0x1c(%esp),%eax
 8048452: 89 44 24 04           mov    %eax,0x4(%esp)
 8048456: c7 04 24 40 85 04 08    movl   $0x8048540,(%esp)
 804845d: e8 8e fe ff ff        call   80482f0 <printf@plt>
 8048462: c9                    leave
 8048463: c3                    ret

08048464 <f1>:
 8048464: 55                    push   %ebp
 8048465: 89 e5                 mov    %esp,%ebp
 8048467: 83 ec 28              sub    $0x28,%esp
 804846a: c7 45 f4 00 00 00 00    movl   $0x0,-0xc(%ebp)
 8048471: 8b 45 0c              mov    0xc(%ebp),%eax
 8048474: 89 44 24 04           mov    %eax,0x4(%esp)
 8048478: 8b 45 08              mov    0x8(%ebp),%eax
 804847b: 89 04 24              mov    %eax,(%esp)
 804847e: e8 0d 00 00 00        call   8048490 <f2>
 8048483: 8b 55 08              mov    0x8(%ebp),%edx
 8048486: 01 d0                 add    %edx,%eax
 8048488: 89 45 f4              mov    %eax,-0xc(%ebp)
 804848b: 8b 45 f4              mov    -0xc(%ebp),%eax
 804848e: c9                    leave
 804848f: c3                    ret

08048490 <f2>:
 8048490: 55                    push   %ebp
```

```
8048491: 89 e5               mov    %esp,%ebp
8048493: 83 ec 10            sub    $0x10,%esp
8048496: c7 45 fc 00 00 00 00    movl   $0x0,-0x4(%ebp)
804849d: 8b 45 08            mov    0x8(%ebp),%eax
80484a0: 0f af 45 0c         imul   0xc(%ebp),%eax
80484a4: 89 45 fc            mov    %eax,-0x4(%ebp)
80484a7: 8b 45 fc            mov    -0x4(%ebp),%eax
80484aa: c9                  leave
80484ab: c3                  ret
80484ac: 66 90               xchg   %ax,%ax
80484ae: 66 90               xchg   %ax,%ax
```

图 1 为该程序的执行到函数 f2 时的栈帧图。

1.该程序运行后计算结果为_____(1 分)；

2.在栈帧图上的“0BC”和“08C”处填上正确的地址值（1 分)；

3.在栈帧图上的“0C0”和“0C4”处填上正确的传递参数值（4 分)；

4.在栈帧图上的“090”和“094”处填上正确的传递参数值（4 分)；

地址	内容
0XBFFFF0E8	OLD EBP
……	……
0DC	
0D8	
0D4	
……	……
0C4	
0C0	
0BC	
0B8	OLD EBP
……	……
0AC	result1
……	……
094	
090	
08C	
(EBP) → 088	OLD EBP
084	result2
……	……
(ESP) → 078	

5.分析题（25 分）

有如下两个 C 语言代码文件

```c
// main_g.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
extern int global_var;
extern void print_global_var();
void handle_signal(int sig) {
    if (sig == SIGUSR1) {
        printf("Received SIGUSR1 signal. Global variable before change: %d\n", global_var);
        global_var = 100;
        printf("Global variable after change: %d\n", global_var);}    }
int main() {
    signal(SIGUSR1, handle_signal);
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(1); } else if (pid == 0) {
        sleep(1);
        kill(getppid(), SIGUSR1);
        exit(0);  } else {
          printf("Parent process waiting for signal.\n");
        pause();
        print_global_var(); }
     return 0; }
```

```c
//globals.c
#include <stdio.h>
int global_var = 42;
void print_global_var() {
    printf("Global variable value: %d\n", global_var);
}
```

对这两个 C 代码文件进行分离编译，分别形成 main_g.o 和 globals.o 文件，再使用 gcc 命令可形成一个可执行文件 test。请问：

1.test 执行的结果是什么？请按照其在屏幕的输出格式书写作答，并请解释为什么有这样的输出。

2.在命令行里面用 gcc 形成可执行文 test 文件时，main_g.o 和 globals.o 在命令行中出现的顺序必须确定吗？为什么？

3.如下图所示，这是哪个文件的哪一部分内容，你怎么看出来的？

4.下图中 call 指令出现的地方很多都有@PLT，这是什么？其作用是什么？

5.为什么在地址 80486af 那一行出现的 call 指令（画横线的部分）没有 PLT 出现？它与其他的 call 相比有什么特点？

6.根据此图，你能大概推算出（画横线部分）07 ff ff ff 是怎么得来的吗？

```
0804861f <main>:
 804861f:       8d 4c 24 04             lea    0x4(%esp),%ecx
 8048623:       83 e4 f0                and    $0xfffffff0,%esp
 8048626:       ff 71 fc                pushl  -0x4(%ecx)
 8048629:       55                      push   %ebp
 804862a:       89 e5                   mov    %esp,%ebp
 804862c:       51                      push   %ecx
 804862d:       83 ec 14                sub    $0x14,%esp
 8048630:       83 ec 08                sub    $0x8,%esp
 8048633:       68 da 85 04 08          push   $0x80485da
 8048638:       6a 0a                   push   $0xa
 804863a:       e8 e1 fd ff ff          call   8048420 <signal@plt>
 804863f:       83 c4 10                add    $0x10,%esp
 8048642:       e8 49 fe ff ff          call   8048490 <fork@plt>
 8048647:       89 45 f4                mov    %eax,-0xc(%ebp)
 804864a:       83 7d f4 00             cmpl   $0x0,-0xc(%ebp)
 804864e:       79 1a                   jns    804866a <main+0x4b>
 8048650:       83 ec 0c                sub    $0xc,%esp
 8048653:       68 ca 87 04 08          push   $0x80487ca
 8048658:       e8 e3 fd ff ff          call   8048440 <perror@plt>
 804865d:       83 c4 10                add    $0x10,%esp
 8048660:       83 ec 0c                sub    $0xc,%esp
 8048663:       6a 01                   push   $0x1
 8048665:       e8 f6 fd ff ff          call   8048460 <exit@plt>
 804866a:       83 7d f4 00             cmpl   $0x0,-0xc(%ebp)
 804866e:       75 2a                   jne    804869a <main+0x7b>
 8048670:       83 ec 0c                sub    $0xc,%esp
 8048673:       6a 01                   push   $0x1
 8048675:       e8 b6 fd ff ff          call   8048430 <sleep@plt>
 804867a:       83 c4 10                add    $0x10,%esp
 804867d:       e8 1e fe ff ff          call   80484a0 <getppid@plt>
 8048682:       83 ec 08                sub    $0x8,%esp
 8048685:       6a 0a                   push   $0xa
 8048687:       50                      push   %eax
 8048688:       e8 e3 fd ff ff          call   8048470 <kill@plt>
 804868d:       83 c4 10                add    $0x10,%esp
 8048690:       83 ec 0c                sub    $0xc,%esp
 8048693:       6a 00                   push   $0x0
 8048695:       e8 c6 fd ff ff          call   8048460 <exit@plt>
 804869a:       83 ec 0c                sub    $0xc,%esp
 804869d:       68 d8 87 04 08          push   $0x80487d8
 80486a2:       e8 a9 fd ff ff          call   8048450 <puts@plt>
 80486a7:       83 c4 10                add    $0x10,%esp
 80486aa:       e8 61 fd ff ff          call   8048410 <pause@plt>
 80486af:       e8 07 ff ff ff          call   80485bb <print_global_var>
 80486b4:       b8 00 00 00 00          mov    $0x0,%eax
 80486b9:       8b 4d fc                mov    -0x4(%ebp),%ecx
 80486bc:       c9                      leave
 80486bd:       8d 61 fc                lea    -0x4(%ecx),%esp
 80486c0:       c3                      ret
```

6．分析题（25 分）

本题考察大家对内存层次结构的组件以及虚拟地址转换为物理地址方式的掌握情况。假设一个使用虚拟存储器的系统具有以下参数：

- 内存是字节可寻址的，内存访问是 1 字节（不是 4 字节）字。
- 虚拟地址为 16 位宽。
- 物理地址为 12 位宽。
- 页大小为 256 字节。
- TLB 有 16 个条目，4 路组相联。
- 高速缓存是 3 路组相联的，具有 4 字节行大小和总共 24 行。

在下表中所有数字均以十六进制表示。TLB、前 32 页的页表和高速缓存的内容如下：

TLB			
Index	Tag	PPN	Valid
0	27	D	1
0	0B	2	0
0	16	3	1
0	3F	9	0
1	1F	C	1
1	3A	8	1
1	22	3	1
1	19	B	1
2	1F	B	0
2	0C	8	0
2	08	9	1
2	12	B	1
3	28	A	1
3	14	0	1
3	3A	9	1
3	07	2	1

Page able					
VPN	PPN	Valid	VPN	PPN	Valid
00	0	1	10	9	1
01	0	1	11	8	1
02	9	0	12	0	0
03	F	1	13	8	0
04	7	0	14	F	0
05	8	0	15	9	1
06	0	1	16	9	0
07	4	1	17	B	1
08	8	1	18	8	1
09	0	0	19	F	1
0A	4	1	1A	3	1
0B	A	0	1B	5	1
0C	B	0	1C	C	1
0D	3	1	1D	D	1
0E	D	0	1E	C	0
0F	0	1	1F	2	1

Cache									
Index	Tag	Valid	Line	Tag	Valid	Line	Tag	Valid	Line
0	62	1	7E813E16	5D	0	6781211C	7B	1	36C461D1
1	2D	1	F2E25610	4C	0	62C2E1A0	26	1	615ECCD0
2	1C	1	21DB6EE7	74	1	D08CEAFF	5F	1	289B1CBB
3	7F	1	53C9FA45	4B	1	D36D3A19	2C	0	F9DEC115
4	3C	1	8F6B663A	4B	1	95EB6C01	2E	0	D3976ECB
5	40	0	9770C6EB	05	0	41FD3F5D	6C	1	F6BFAE6E
6	3F	1	C4C13D4F	5F	1	FC79DCE1	1E	0	99B624A8
7	49	1	DE8C2265	78	1	D99937C5	11	1	72C2AF91

第一部分：基本参数（每空 1 分，共 9 分）

1、下图显示了虚拟地址的格式。

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0



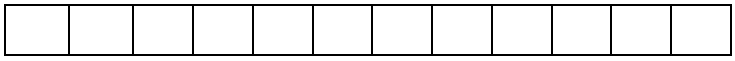
请指出以下字段分别在图中的哪些位？

VPO：_____VPN：_____

TLBI：_____TLBT：_____

2、下图显示了物理地址的格式。

11 10 9 8 7 6 5 4 3 2 1 0



请指出以下字段分别在图中的哪些位？

PPO：_____PPN：_____

CO：_____CI：_____

CT：_____

第二部分：地址翻译（16 分）

对于给定的虚拟地址，请指明访问的 TLB 条目和物理地址和以十六进制返回的高速缓存字节值。指明 TLB 是否未命中、是否发生页面错误以及是否发生高速缓存未命中。

如果高速缓存未命中，请输入“N/A”作为“返回的高速缓存字节”。如果存在缺页，请在“PPN”中输入“N/A”，并将 C 部分和 D 部分留空。

虚拟地址：0xEB6E

A、虚拟地址（2 分，每格一位，填写 0 或 1）

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

B、地址翻译（每空 1 分，答案以十六进制表示）

参数	**值**	**参数**	**值**
VPN	0x	**TLB 命中？（是/否）**	
TLBI	0x	**缺页？（是/否）**	
TLBT	0x	**PPN**	0x

C、物理地址（2 分，每格一位，填写 0 或 1）

11 10 9 8 7 6 5 4 3 2 1 0

D、物理内存引用（每空 1 分，答案以十六进制表示）

参数	**值**	**参数**	**值**
CO	0x	**物理地址**	0x
CI	0x	**高速缓存命中？（是/否）**	
CT	0x	**高速缓存返回的字节值**	0x